

Computational Implementation and Practical Application

Technology Stack and Design Rationale

To translate the mathematical model into a functional, accessible tool, we developed a full-stack web application named *IEcare*. The application was built using Next.js (a React-based framework) with TypeScript, chosen for its type safety, which reduces the likelihood of runtime errors when handling the numerous numerical parameters in the model. The core optimisation engine relies on the `javascript-lp-solver` library, a lightweight implementation of the Simplex method that solves linear programming problems directly in the browser or on the server. This architectural decision was deliberate: by running the solver server-side via Next.js API routes, we avoid requiring users to install any software — hospital administrators can simply open a URL, enter their parameters, and receive optimal resource allocations instantly.

The front-end interface was designed with practical usability in mind, featuring clearly labelled input fields organised into four logical groups — Budget and Costs, Staffing Ratios, Space and Operations, and Minimum Staffing — each accompanied by contextual tooltips explaining what each parameter represents and how it should be estimated. Results are displayed with visual progress bars showing constraint utilisation, making it immediately clear which resources are near capacity and which have slack. The application also supports exporting results to Excel, enabling hospital planners to integrate the outputs into existing reporting workflows.

From Mathematical Model to Code

The central challenge in implementation was faithfully encoding the mathematical model — as developed in the preceding section — into a format the LP solver can process. The mathematical formulation seeks to:

Maximise P (concurrent patients)

Subject to: - Doctor capacity: $(3 / p_d) \times P \leq d$ - Nurse capacity: $(3 / p_n) \times P \leq n$ - Equipment capacity: $e_p \times P \leq e$ - Bed capacity: $b_p \times P \leq b$ - Budget: $C_d \cdot d + C_n \cdot n + C_e \cdot e + C_b \cdot b \leq B$ - Space: $A_e \cdot e + A_b \cdot b \leq A_T$ - Minimum staffing: $d \geq d_min, n \geq n_min$ - Non-negativity and integrality: $P, d, n, e, b \in \mathbb{Z}^+$

The LP solver requires this to be expressed as a structured model object. In the implementation, each constraint becomes a named row in a constraint matrix, and each decision variable (P, d, n, e, b) specifies its coefficient in every constraint it participates in. For example, the doctor capacity constraint — which states that the number of doctor FTEs needed $(3/p_d \times P)$ must not exceed the number hired (d) — is encoded by assigning P a coefficient of $3/p_d$ in the `doctor_cap` row and assigning d a coefficient of -1 in the same row, with the constraint bounded at zero. This formulation ensures that the solver simultaneously determines the optimal patient count and the corresponding resource levels.

A critical detail in the implementation is the 24/7 shift-coverage multiplier. Hospitals operate continuously, requiring three 8-hour shifts per day. If one doctor can manage p_d patients during a single shift, then covering one patient around the clock requires $3/p_d$ doctor FTEs. For instance, if $p_d = 3$ (one doctor handles 3 patients per shift), then each patient requires $3/3 = 1.0$ doctor FTE for full-day coverage. This multiplier is applied identically to nursing staff. Equipment and beds, being physical assets that do not rotate with shifts, use their ratios directly (e_p and b_p) without the $\times 3$ factor.

The budget constraint aggregates all resource costs linearly: each unit of doctors, nurses, equipment, and beds contributes its respective annual cost (C_d, C_n, C_e, C_b) to the total, which must not exceed the annual budget B . Similarly, the space constraint sums the floor area consumed by equipment (A_e per unit) and beds (A_b per unit), which must fit within the total available clinical space A_T . Notably, staff do not consume modelled floor space — only physical assets (monitoring stations and beds) occupy ward area.

Integer Solution Search

The LP solver operates in continuous space, meaning it may return fractional solutions such as $P = 14.73$ patients or $d = 8.2$ doctors. In practice, a hospital

cannot admit 0.73 of a patient or employ 0.2 of a doctor. Therefore, the implementation includes a dedicated integer solution search that guarantees a feasible whole-number result.

The approach works as follows. First, the continuous LP relaxation is solved to obtain the theoretical maximum P . *This value serves as an upper bound — no integer solution can exceed it. The algorithm then iterates downward from $\lfloor P \rfloor$ (the largest integer not exceeding P^*), and for each candidate patient count p , it computes the exact integer resources required:*

- $d = \max(d_{\min}, \lceil (3/p_d) \times p \rceil)$
- $n = \max(n_{\min}, \lceil (3/p_n) \times p \rceil)$
- $e = \lceil e_p \times p \rceil$
- $b = \lceil b_p \times p \rceil$

The ceiling function ($\lceil \cdot \rceil$) ensures rounding up — if 14 patients require 7.17 monitoring stations, we must purchase 8. The $\max()$ function ensures that regulatory minimum staffing levels are always met, even if the patient count would otherwise require fewer staff.

For each candidate, the algorithm checks two feasibility conditions: (1) the total cost $C_d \cdot d + C_n \cdot n + C_e \cdot e + C_b \cdot b$ does not exceed the budget B , and (2) the total space $A_e \cdot e + A_b \cdot b$ does not exceed A_T . The first candidate p that satisfies both conditions is the optimal integer solution, and the search terminates. Because the search proceeds from the highest possible value downward, the first feasible solution found is guaranteed to be optimal.

This approach is computationally efficient for our problem size — the gap between the LP optimum and the integer optimum is typically small (at most a few patients), so the loop executes only a handful of iterations. For larger-scale problems with many more decision variables, a formal Mixed-Integer Programming (MIP) solver would be more appropriate, but for our four-resource model, the enumeration approach is both exact and instantaneous.

The implementation also handles edge cases rigorously. If the minimum staffing cost alone ($C_d \cdot d_{\min} + C_n \cdot n_{\min}$) exceeds the budget, the model returns an infeasibility diagnosis with a specific explanation rather than a silent failure. Similarly, if the floor space required for a single patient's equipment and bed exceeds the total available area, the solver reports this constraint violation explicitly. These diagnostic messages are designed to

guide users toward actionable adjustments — for example, "Minimum staffing alone costs €520,000 but budget is only €400,000. Reduce minimum staff requirements or increase budget."

Sensitivity Analysis Engine

Beyond finding a single optimal solution, a critical question for hospital administrators is: *how robust is this result?* If the budget changes by 10%, does the optimal capacity shift dramatically or barely at all? Which constraints are truly limiting, and where would an additional euro of investment yield the greatest return? The sensitivity analysis module addresses these questions systematically.

The engine performs five distinct analyses, each requiring hundreds of solver evaluations (approximately 1,500 in total):

Parameter Sweeps. Each input parameter is varied individually from -50% to $+50\%$ of its baseline value in incremental steps, while all other parameters remain fixed. For each variation, the solver re-optimises, producing a curve that shows how patient capacity responds to changes in that parameter. These curves reveal whether relationships are smooth and linear or contain sharp transitions where a new constraint becomes binding.

Tornado Diagram. To quickly identify the most influential parameters, the engine evaluates the impact of a $\pm 20\%$ change in each parameter on the optimal patient count. The results are ranked by sensitivity (the absolute difference between the $+20\%$ and -20% outcomes). Parameters at the top of the tornado chart — those producing the widest bars — are the ones where estimation accuracy matters most and where investment would have the greatest impact.

Approximate Shadow Prices (Marginal Values). For each major constraint (budget, space, doctor staffing, nurse staffing), the engine computes a finite-difference approximation of the shadow price — the rate at which the objective function improves per unit relaxation of that constraint. Concretely, it bumps each constraint limit by a small epsilon (1% of its current value), resolves, and calculates $\Delta P / \Delta \epsilon$. A high marginal value (e.g., "+€1 budget $\rightarrow$ +0.003 patients") indicates that the constraint is actively limiting capacity, while a near-zero value means the constraint has slack and relaxing it would

yield no benefit. This information directly informs capital allocation decisions: if the budget shadow price is high but the space shadow price is zero, the hospital should prioritise securing additional funding over expanding floor area.

Budget–Space Heatmap. Since budget and space are typically the two most impactful macro-constraints, the engine generates a two-dimensional grid varying both simultaneously. Each cell in the resulting heatmap shows the optimal patient count for that (Budget, Space) combination, colour-coded from dark (few patients) to bright blue (many patients). This visualisation helps administrators understand trade-offs — for example, whether doubling the budget alone is sufficient or whether physical expansion is also necessary.

Bottleneck Breakpoints. Finally, the engine scans for parameter thresholds where the binding constraint shifts from one resource to another. For instance, at a budget of €5M the bottleneck might be "Annual Budget," but at €6M, space becomes the new limiter. Identifying these transition points helps planners anticipate which constraint will become critical as conditions evolve over time.

All sensitivity results can be exported to a multi-sheet Excel workbook, with dedicated tabs for the tornado data, each parameter sweep, shadow prices, heatmap values, and breakpoint thresholds — providing a complete analytical package that can be shared with stakeholders who may not interact with the web application directly.